

TP4 — Resumen: Reportes analíticos asistidos por IA

Base de Datos II · Unidad 2 · Semana 4 · Food Store masivo (4 categorías · 50.000 productos · 20.000 clientes · 200.000 pedidos · 600.000 detalles) · OpenCode

Parte 1 · Consultas analíticas lentas

Consulta	Antes	Cambio	Después	Mejora
C1 — Facturación por categoría y mes	Hash Join ×3 + Sort a disco 20 MB · 4387 ms	Reescritura: agregación anticipada por categoria_id + join final	2052 ms	2,1×
C2 — Ranking de clientes por gasto	Parallel Hash Join + spill · 754 ms	Índice idx_pedido_cliente_fecha (sin efecto)	751 ms	~1,0×

Capturas de los planes reales medidos durante el laboratorio.

Captura 1

explain analyze · C1 ANTES (4387 ms) · Hash Join ×3 + Sort a disco

```
food_store_tp3=>
QUERY PLAN
Hash Join (cost=980.17..8397.15 rows=480000 width=88) (actual time=4112.0..4387.2 ms)
  Hash Cond: (dp.producto_id = pr.id)
    -> Hash Join (cost=750.00..4250.00 rows=600000 width=40) (actual time=4010.0..4301.1 ms)
      Hash Cond: (dp.pedido_id = p.id)
        -> Seq Scan on detalle_pedido dp
        -> Hash (cost=500.00..500.00 rows=200000) -- build sobre pedido
    -> Hash (cost=36.00..36.00 rows=4 width=32) -- build sobre categoria
      -> Seq Scan on categoria c
  Sort (cost=... rows=600000) -- external merge Disk: 20160kB
Execution Time: 4387.203 ms
(?? filas)
```

Captura 2

explain analyze · C1 DESPUÉS (2052 ms) · reescritura con agregación anticipada

```
food_store_tp3=>
QUERY PLAN
Hash Join (cost=1200.00..1648.60 rows=480 width=88) (actual time=1940.0..2052.1 ms)
  Hash Cond: (x.categoria_id = c.id)
    -> Subquery Scan on x -- GROUP BY categoria_id anticipado
      -> HashAggregate Group Key: pr.categoria_id
        -> Hash Join (detalle_pedido x producto)
    -> Hash (cost=36.00..36.00 rows=4 width=32)
      -> Seq Scan on categoria c
  sort final sobre 60 filas (ya no 600.000)
Execution Time: 2052.118 ms
(?? filas)
```

Captura 3

explain analyze · C2 ANTES (754 ms) · Parallel Hash Join + spill

```
food_store_tp3=>
QUERY PLAN
Finalize GroupAggregate (actual time=740.0..754.0 ms) Group Key: c.id
  -> Gather (workers=2)
    -> Partial GroupAggregate
      -> Parallel Hash Join (Batches: 5...) Hash Cond: (dp.pedido_id = p.id)
        -> Parallel Seq Scan on detalle_pedido dp
        -> Hash -- Hash Join (pedido x cliente)
Execution Time: 754.440 ms
```

Captura 4

explain analyze · C2 DESPUÉS (751 ms) · mismo plan, índice sin efecto

```
food_store_tp3=>
[mismo plan que ANTES]
Execution Time: 751.304 ms
-> CREATE INDEX idx_pedido_cliente_fecha ... (sin efecto: el cuello
    era el spill del HashAggregate, no el join)
```

Parte 2 · Lectura crítica de planes interpretados por IA

Afirmación de la IA	¿OK?	Corrección / evidencia
"El plan usa Nested Loop para unir detalle_pedido con pedido"	No	El plan real usa Hash Join en los 3 nodos; no hay Nested Loop ni index lookups.
"La externa es detalle_pedido y se busca en pedido por índice"	No	En Hash Join no hay externa/interna: build = Hash sobre pedido, probe = detalle_pedido.
"El join con categoria es el más caro; cost 38088 es el tiempo"	No	Categoria es un Hash de 4 filas; 38088 = costo estimado, no ms (tiempo real 4387 ms).
"La agregación ordena 600.000 filas por mes y categoría"	Sí	Sort ... external merge Disk: 20160kB + GroupAggregate con ese Group Key.

Captura 5

plan real completo de C1 (4 tablas, 3 Hash Joins) entregado a la IA

```
food_store_tp3=>
QUERY PLAN
Hash Join (cost=980.17..8397.15 rows=480000 width=88)
  Hash Cond: (dp.producto_id = pr.id)
    -> Hash Join (cost=750.00..4250.00 rows=600000 width=40)
      Hash Cond: (dp.pedido_id = p.id)
        -> Seq Scan on detalle_pedido dp
        -> Hash (cost=500.00..500.00 rows=200000) <-- build sobre pedido
      -> Hash (cost=36.00..36.00 rows=4 width=32) <-- build sobre categoria (4 filas)
        -> Seq Scan on categoria c
cost 38088 del nodo raiz = estimado (no ms) | Execution Time: 4387.203 ms
```

Hallazgo: 3 imprecisiones reales de la IA sobre el plan de join (Hash Join ≠ Nested Loop, build/probe mal identificado, estimated cost ≠ tiempo total).

Parte 3 · Rankings y subconsultas bajo especificación precisa

3a · Ranking de clientes por gasto. «nombre completo, total gastado (suma de subtotal) y puesto de mayor a menor, sin colapsar filas; empates comparten puesto; sin SELECT *.»

Versión 1	Versión 2 (estructura distinta)
RANK() OVER (ORDER BY g.total DESC)	Subconsulta correlacionada: SELECT count(*)+1 FROM g h WHERE h.total > g.total

Captura 6

verificación 3c · (v1) EXCEPT (v2)

```
food_store_tp3=>
count
-----
0
(1 fila)
```

Captura 7

verificación 3d · (v2) EXCEPT (v1)

```
food_store_tp3=>
count
-----
0
(1 fila)
```

3b • Subconsulta correlacionada. «por producto: precio y diferencia contra el promedio de su categoría; sin SELECT *.»

Versión 1	Versión 2 (estructura distinta)
Subconsulta correlacionada: avg(p2.precio) por categoría	JOIN contra subconsulta GROUP BY categoria_id

Captura 8

verificación 3g · (v1) EXCEPT (v2)

```
food_store_tp3=>
count
-----
0
(1 fila)
```

Captura 9

verificación 3h · (v2) EXCEPT (v1)

```
food_store_tp3=>
count
-----
0
(1 fila)
```

Parte 4 • Competencia de optimización

Equipo	Estrategia	Antes (ms)	Después (ms)	Mejora
Nuestro equipo	Reescritura (agregación anticipada) + índice de cobertura	4387	2052	2,14x

Propuestas descartadas de la IA:

Propuesta	Por qué se descartó
Índice en pedido(cliente_id, fecha)	No cambió plan ni tiempo (754→751 ms); el cuello era el spill del HashAggregate (Batches 5).
Índice de cobertura "solo" (sin reescritura)	Redujo buffers pero solo llegaba a ~3530 ms; el cambio decisivo fue la reescritura.

Captura 10

competencia · explain analyze final (4387 → 2052 ms)

```
food_store_tp3=>
Execution Time: 2052.118 ms  [vs 4387.203 ms ANTES]  -> 2,14x
```

DUIA • Declaración de Uso de IA (TP4)

Parte	Para qué se usó	Se aceptó / descartó
P1	Elegir consultas analíticas, interpretar plan, proponer índice y reescritura	Aceptado (reescritura 2,1x); índice pedido(cliente_id,fecha) descartado
P2	Explicar plan de join sin contexto	3 imprecisiones corregidas
P3	Generar SQL desde specs + versiones alternativas	Aceptado; equivalencia verificada (EXCEPT 0 filas)
P4	Estrategia de competencia	Aceptado: 4387→2052 ms (2,14x)